```
  def inspect
    "#<Event #{name.inspect} (capacity: #{capacity})>"
  end
end
```

Now it's time to add the first specs.

When No Tickets Have Been Sold

Put the following example group after your class:

12-creating-custom-matchers/exercises/custom_matcher/spec/event_matchers_spec.rb

```
RSpec.describe '`have_no_tickets_sold` matcher' do
  example 'passing expectation' do
    art_show = Event.new('Art Show', 100)

    expect(art_show).to have_no_tickets_sold
  end

  example 'failing expectation' do
    art_show = Event.new('Art Show', 100)
    art_show.purchase_ticket_for(:a_friend)

    expect(art_show).to have_no_tickets_sold
  end
end
```

Go ahead and run your spec file. Both examples should fail, because the have_no_tickets_sold matcher doesn't exist yet.

Implement this matcher using the matcher DSL. When your match logic is correct, you'll have one passing and one failing example. Then, you can turn your attention to providing a good failure message.

Selling Out

Now that we have a handle on unsuccessful ticket sales, let's test what happens for a sold-out concert. Add the following example group:

12-creating-custom-matchers/exercises/custom_matcher/spec/event_matchers_spec.rb

```
RSpec.describe '`be_sold_out` matcher' do
  example 'passing expectation' do
    u2_concert = Event.new('U2 Concert', 10_000)
    10_000.times { u2_concert.purchase_ticket_for(:a_fan) }

    expect(u2_concert).to be_sold_out
  end
```